Listing 4.6 `autogen_start.py`

```
from autogen import ConversableAgent, UserProxyAgent, config_list_from_json


   config_list = config_list_from_json(
        env_or_file="OAI_CONFIG_LIST")      #1

   assistant = ConversableAgent(
        "agent",
        llm_config={"config_list": config_list})      #2

   user_proxy = UserProxyAgent(      #3
        "user",
        code_execution_config={
            "work_dir": "working",
            "use_docker": False,
        },
        human_input_mode="ALWAYS",
        is_termination_msg=lambda x: x.get("content", "")
        .rstrip()
        .endswith("TERMINATE"),      #4
    )
    user_proxy.initiate_chat(assistant, message="write a solution
↪ for fizz buzz in one line?")     #5
```

#1 Loads your LLM configuration from the JSON file OAI_CONFIG_LIST
#2 This agent talks directly to the LLM.
#3 This agent proxies conversations from the user to the assistant.
#4 Setting the termination message allows the agent to iterate.
#5 A chat is initiated with the assistant through the user_proxy to complete a task.

Run the code by running the file in VS Code in the debugger (F5). The code in listing 4.6 uses a simple task to demonstrate code writing. Listing 4.7 shows a few examples to choose from. These coding tasks are also some of the author's regular baselines to assess an LLMs' strength in coding.

Listing 4.7 Simple coding task examples

```
write a Python function to check if a number is prime
code a classic sname game using Pygame                  #1
code a classic asteroids game in Python using Pygame  #1
```

#1 To enjoy iterating over these tasks, use Windows Subsystem for Linux (WSL) on Windows, or use Docker.

After the code starts in a few seconds, the assistant will respond to the proxy with a solution. At this time, the proxy will prompt you for feedback. Press Enter, essentially giving no feedback, and this will prompt the proxy to run the code to verify it operates as expected.